



Optimalidad de Pareto¹

Miguel Ricardo Calderón Serpa 2220967

Karen Julieth Bermudez Calderón 2220950

Nazhly Zharith Florez Martinez 2220959

Daniela Alejandra Mesa Mesa 2211059

1 Introducción

La optimización matemática es una disciplina que busca encontrar la mejor solución posible dentro de un conjunto de alternativas, restricciones y objetivos definidos. Precizando en este concepto, un problema de optimización busca minimizar o maximizar una función objetivo $f(x)$, donde x es un vector de variables de decisión sujeto a restricciones $g_i(x) \leq 0$ y $h_j(x) = 0$. La optimización multiobjetivo busca encontrar solución a problemas con más de una función objetivo y múltiples objetivos a considerar. Este trabajo se centrará en estudiar este tipo de problemas teniendo como enfoque la Optimalidad de Pareto, los métodos que se pueden usar dependiendo de cómo sea la función (diferenciable, no diferenciable, convexa, no convexa), y el problema de Markowitz con soluciones en dicho tema.

El economista Vilfredo Pareto la define como “el estado en el que ningún individuo o criterio puede mejorar sin perjudicar a otro”. Junto a esta definición se presenta también el frente de Pareto, que representa el conjunto de soluciones no dominadas. En la primera parte se presentan algunas definiciones y conceptos de la Optimalidad de Pareto. Más adelante se abordan teoremas cuando las funciones objetivo son diferenciables, no diferenciables, y sus respectivas condiciones de primer orden y condiciones de segundo orden. Luego se exponen algunos métodos relevantes: el método Proximal Multiobjetivo Bundle (MPB), ε -restricción, método de la suma ponderada y el NIMBUS, junto con sus algoritmos. Finalmente, se aborda el problema Markowitz con solución en diferentes métodos, gráficos y su respectivo código.

2 Optimización multiobjetivo

Un problema de optimización multiobjetivo (POM) es de la forma:

$$\begin{aligned} &\text{Minimizar(max)} \quad F(x) = [f_1(x), f_2(x), \dots, f_k(x)]^T \\ &\text{Sujeto a} \quad x \in \Omega \end{aligned}$$

Donde $k \geq 2$ es el número de funciones objetivo $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$, y $x = (x_1, x_2, \dots, x_n)$ es el vector de variables de decisión, que pertenece a la región factible $\Omega \subseteq \mathbb{R}^n$.

La palabra 'Minimizar' indica que se quiere minimizar todas las funciones objetivo de forma simultánea. En el caso de que no exista conflicto entre las funciones, podemos determinar una solución tal que todas

¹21 de mayo, 2025

las funciones $f_i(x)$ alcancen su mínimo en este; Sin embargo este caso resulta trivial pues se reduce a un problema de optimización clásica. Es por ello que se asume no existe alguna solución que sea óptima simultáneamente en cada una de las funciones objetivo.

Es por lo explicado anteriormente, que la solución de un problema PMO resulta ser mas un concepto que una definición. Es por ello que se han diseñado distintos métodos para comprender y atacar este tipo de problemas.

3 Optimalidad de Pareto

3.1. Óptimo de Pareto

Un punto $x^* \in X$ es óptimo de Pareto si, y solo si no existe otro punto $x \in X$, tal que $f_i(x) \leq f_i(x^*)$ para todo $i = 1, \dots, k$ y $f_j(x) < f_j(x^*)$ para al menos un índice j

3.2. Frente de Pareto

El frente de Pareto es el conjunto

$$PF = \{F(x) \in \mathbb{R}^k \mid x \in X \text{ es un óptimo de Pareto}\}$$

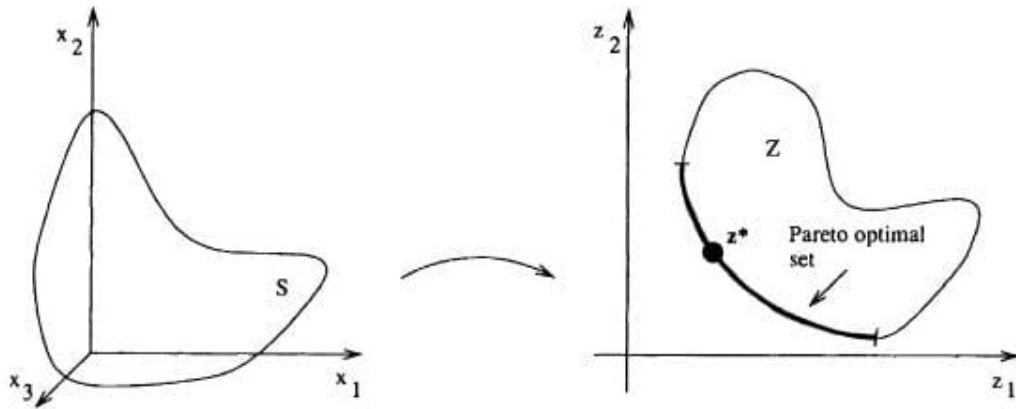


Figura 1: Ilustración de una región factible $S \subset \mathbb{R}^3$ y su imagen, una región objetivo factible $Z \subset \mathbb{R}^2$. La línea gruesa contiene todos los vectores objetivo óptimos de Pareto. El vector z^* es un ejemplo de ellos.

3.3. Óptimo de Pareto debil

Un punto, $x^* \in X$ es óptimo de Pareto debil si, y solo si, no existe otro punto $x \in X$, tal que $f_i(x) < f_i(x^*)$ para todo $i = 1, \dots, k$

3.4. Óptimo de Pareto propio

Un punto, $x^* \in X$, es óptimo de Pareto propio si es óptimo de Pareto y además existe un número real $M > 0$ tal que para cada $f_i(x)$ y cada $x \in X$ que satisface $f_i(x) < f_i(x^*)$, entonces existe al menos un $f_j(x)$

tal $f_j(x^*) < f_j(x)$ y $\frac{f_i(x^*) - f_i(x)}{f_j(x) - f_j(x^*)} \leq M$. Si un punto óptimo de Pareto no es propio, se dice impropio.

Teorema 3.4.1

Sea el problema de optimización multiobjetivo convexo. Entonces, toda solución óptima de Pareto local también es óptima de Pareto global.

Sea $\mathbf{x}^* \in S$ localmente óptima de Pareto. Entonces, existen $\delta > 0$ y un vecindario $B(\mathbf{x}^*, \delta)$ de $\mathbf{x}^*$ tal que no existe $\mathbf{x} \in S \cap B(\mathbf{x}^*, \delta)$ para la cual:

$$f_i(\mathbf{x}) \leq f_i(\mathbf{x}^*) \quad \text{para todo } i = 1, \dots, k$$

y para al menos un índice j se cumpla:

$$f_j(\mathbf{x}) < f_j(\mathbf{x}^*).$$

Supongamos que $\mathbf{x}^*$ no es globalmente óptima de Pareto. En este caso, existe otro punto $\mathbf{x}^o \in S$ tal que:

$$f_i(\mathbf{x}^o) \leq f_i(\mathbf{x}^*) \quad \text{para todo } i = 1, \dots, k$$

y

$$f_j(\mathbf{x}^o) < f_j(\mathbf{x}^*) \quad \text{para algún } j.$$

Definamos:

$$\tilde{\mathbf{x}} = \beta \mathbf{x}^o + (1 - \beta) \mathbf{x}^*, \quad \text{donde } 0 < \beta < 1$$

se elige tal que $\tilde{\mathbf{x}} \in B(\mathbf{x}^*, \delta)$. La convexidad de S implica que $\tilde{\mathbf{x}} \in S$.

Por la convexidad de las funciones objetivo y empleando (2.2.1), obtenemos:

$$f_i(\tilde{\mathbf{x}}) \leq \beta f_i(\mathbf{x}^o) + (1 - \beta) f_i(\mathbf{x}^*) \leq \beta f_i(\mathbf{x}^*) + (1 - \beta) f_i(\mathbf{x}^*) = f_i(\mathbf{x}^*) \quad \forall i = 1, \dots, k.$$

Dado que $\mathbf{x}^*$ es localmente óptima de Pareto y $\tilde{\mathbf{x}} \in B(\mathbf{x}^*, \delta)$, se debe cumplir:

$$f_i(\tilde{\mathbf{x}}) = f_i(\mathbf{x}^*) \quad \text{para todo } i.$$

Además:

$$f_i(\mathbf{x}^*) \leq \beta f_i(\mathbf{x}^o) + (1 - \beta) f_i(\mathbf{x}^*) \quad \forall i = 1, \dots, k.$$

Como $\beta > 0$, dividiendo entre β :

$$f_i(\mathbf{x}^*) \leq f_i(\mathbf{x}^o) \quad \forall i.$$

Por (3.1), $f_j(\mathbf{x}^*) > f_j(\mathbf{x}^o)$ para algún j , lo que genera una contradicción. Por tanto, $\mathbf{x}^*$ es globalmente óptima de Pareto.

4 Condiciones de Optimalidad de Pareto

4.1. Condiciones de Optimalidad Diferenciable

Es importante tener en cuenta que, el problema se restringirá al caso en que el conjunto de decisiones este contenido en un espacio euclidiano de dimensión finita y además serán continuamente diferenciables, se considerarán problemas de la forma

Problema General Diferenciable (Problema 4.1)

Minimizar $\{f_1(x), f_2(x), \dots, f_k(x)\}$
sujeto a $x \in S = \{x \in \mathbb{R}^n \mid g(x) = (g_1(x), \dots, g_m(x))^T \leq 0\}$.

Denotamos el conjunto de restricciones activas en un punto x^* por:

$$J(x^*) = \{j \in \{1, \dots, m\} \mid g_j(x^*) = 0\}$$

4.1.1. Condiciones de Primer Orden

Teorema 4.1.1.1 (Condición necesaria de Fritz John para la optimalidad de Pareto)

Sean las funciones objetivo y de restricción (problema 4.1) continuamente diferenciables en un vector de decisión $x \in S$. Una condición necesaria para que x sea un óptimo de Pareto es que existen vectores $\lambda \in \mathbb{R}^n$, con $\lambda > 0$, y $\mu \in \mathbb{R}^m$, con $\mu \geq 0$, tales que $(\lambda, \mu) \neq (0, 0)$ y se cumple:

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*) + \sum_{j=1}^m \mu_j \nabla g_j(x^*) = 0 \quad (1)$$

$$\mu_j g_j(x^*) = 0 \quad \text{Para todo } j = 1, \dots, m. \quad (2)$$

Corolario 4.1.1.2 (Condición necesaria de Fritz John para la optimalidad debil de Pareto)

La condición del Teorema 4.1.1 también es necesaria para que un vector de decisión $x^* \in S$ sea débilmente óptimo de Pareto.

Definición 4.1.1.3 (Calificación de restricciones de Kuhn-Tucker)

Sean las funciones de restricción g_j del problema (4.1) continuamente diferenciables en $x^* \in S$. El problema satisface la calificación de restricciones de Kuhn-Tucker en x^* si, para cualquier $d \in \mathbb{R}^n$ tal que

$$\nabla g_j(x^*)^T d \leq 0 \quad \text{para todo } j \in J(x^*),$$

existe una función $a : [0, 1) \rightarrow \mathbb{R}^n$, continuamente diferenciable en 0, y un escalar real $\alpha > 0$, tales que:

1. $a(0) = x^*$,
2. $g(a(t)) \leq 0$ para todo $0 \leq t \leq 1$,
3. $a'(0) = \alpha d$.

Teorema 4.1.1.4 (Condición necesaria de KKT para la optimalidad de Pareto)

Sean satisfechas las suposiciones del Teorema 4.1.1 junto con la condición de cualificación de restricciones de Kuhn-Tucker. Entonces, el Teorema 4.1.1 sigue siendo válido con la adición de que $\lambda \geq 0$.

Corolario 4.1.1.5 (Condición necesaria de Karush-Kuhn-Tucker para la optimalidad débil de Pareto) La condición del Teorema 4.1.4 también es necesaria para que un vector de decisión $x^* \in S$ sea débilmente óptimo de Pareto.

Teorema 4.1.1.6 (Condición suficiente de Karush-Kuhn-Tucker para la optimalidad de Pareto)

Sean las funciones objetivo y de restricción del problema (4.1) convexas y continuamente diferenciables en un vector de decisión $x^* \in S$.

Una condición suficiente para que x^* sea óptimo de Pareto es que existan multiplicadores $0 < \lambda \in \mathbb{R}^k$ y $0 \leq \mu \in \mathbb{R}^m$ tales que:

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*) + \sum_{j=1}^m \mu_j \nabla g_j(x^*) = 0,$$

$$\mu_j g_j(x^*) = 0 \quad \text{para todo } j = 1, \dots, m.$$

Teorema 4.1.1.7 (Condición suficiente de Karush-Kuhn-Tucker para la optimalidad de Pareto débil)

La condición del Teorema 4.1.6 es suficiente para que un vector de decisión $x^* \in S$ sea óptimo de Pareto débil, para $0 \leq \lambda \in \mathbb{R}^k$ con $\lambda \neq 0$.

4.1.2. Condiciones de Segundo Orden

Definición 4.1.2.1

Se dice que un punto $x^* \in S$ es un punto regular si los gradientes de las restricciones activas en x^* son linealmente independientes.

Teorema 4.1.2.2 (Condición necesaria de segundo orden para la optimalidad de Pareto)

Sean las funciones objetivo y de restricción del problema (3.1.1) dos veces continuamente diferenciables en un vector de decisión regular $x^* \in S$. Una condición necesaria para que x^* sea óptimo de Pareto es que existan vectores $0 \leq \lambda \in \mathbb{R}^k$, con $\lambda \neq 0$, y $0 \leq \mu \in \mathbb{R}^m$ tales que:

1.

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*) + \sum_{j=1}^m \mu_j \nabla g_j(x^*) = 0,$$

2.

$$\mu_j g_j(x^*) = 0 \quad \text{para todo } j = 1, \dots, m,$$

3.

$$d^T \left(\sum_{i=1}^k \lambda_i \nabla^2 f_i(x^*) + \sum_{j=1}^m \mu_j \nabla^2 g_j(x^*) \right) d > 0,$$

para todo $d \in \{0 \neq d \in \mathbb{R}^n \mid \nabla f_i(x^*)^T d \leq 0 \text{ para todo } i = 1, \dots, k, \nabla g_j(x^*)^T d \leq 0 \text{ para todo } j \in J(x^*)\}$

o bien para todo

$$d \in \{0 \neq d \in \mathbb{R}^n \mid \nabla g_j(x^*)^T d = 0 \text{ para todo } j \in J^+(x^*), \nabla g_j(x^*)^T d \leq 0 \text{ para todo } j \in J(x^*) \setminus J^+(x^*)\},$$

donde $J^+(x^*) = \{j \in J(x^*) \mid \mu_j > 0\}$.

4.2. Condiciones de Optimalidad No Diferenciable

4.2.1. Condiciones de Primer Orden

Teorema 4.2.1.1 (Condición necesaria de Fritz John para la optimalidad de Pareto)

Sean las funciones objetivo y las funciones de restricción del problema (3.2.1) localmente Lipschitzianas en un punto $x^* \in S$.

Una condición necesaria para que el punto x^* sea óptimo de Pareto es que existan multiplicadores $\lambda \in \mathbb{R}^k$, $\mu \in \mathbb{R}^m$, con $\lambda \geq 0$, $\mu \geq 0$ y $(\lambda, \mu) \neq (0, 0)$, tales que:

$$0 \in \sum_{i=1}^k \lambda_i \partial f_i(x^*) + \sum_{j=1}^m \mu_j \partial g_j(x^*)$$

y

$$\mu_j g_j(x^*) = 0 \quad \text{para todo } j = 1, \dots, m.$$

Corolario 4.2.1.2 (Condición necesaria de Fritz John para la optimalidad de Pareto débil)

La condición del Teorema 4.2.1.1 también es necesaria para que un vector de decisión $x^* \in S$ sea óptimo de Pareto débil.

Teorema 4.2.1.3 (Condición necesaria de Karush-Kuhn-Tucker para la optimalidad de Pareto)

Sean satisfechas las hipótesis del Teorema 4.2.1.1 mediante la condición de calificación de restricciones de Cottle. Entonces, el Teorema 4.2.1.1 sigue siendo válido con la adición de que $\lambda \neq 0$.

Teorema 4.2.1.5 (Condición suficiente de Karush-Kuhn-Tucker para la optimalidad de Pareto)

Sea el problema (4.1) convexo. Una condición suficiente para que un vector de decisión $x^* \in S$ sea óptimo de Pareto es que existan multiplicadores $0 < \lambda \in \mathbb{R}^k$ y $0 \leq \mu \in \mathbb{R}^m$ tales que

$$\begin{cases} 0 \in \sum_{i=1}^k \lambda_i \partial f_i(x^*) + \sum_{j=1}^m \mu_j \partial g_j(x^*), \\ \mu_j g_j(x^*) = 0, \quad \text{para todo } j = 1, \dots, m. \end{cases}$$

4.2.2. Condiciones de Segundo Orden

Teorema 4.2.2.1 (Condición necesaria de segundo orden para la optimalidad de Pareto)

Sean las funciones objetivo del problema (4.1) funciones de clase $C^{1,1}$ y sus funciones de restricción dos veces continuamente diferenciables en un vector de decisión $x^* \in S$.

Una condición necesaria para que x^* sea óptimo de Pareto es que existan vectores $0 \leq \lambda \in \mathbb{R}^k$ y $0 \leq \mu \in \mathbb{R}^m$, tales que $(\lambda, \mu) \neq (0, 0)$ y se cumpla:

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*) + \sum_{j=1}^m \mu_j \nabla g_j(x^*) = 0,$$

$$\mu_j g_j(x^*) = 0, \quad \text{para todo } j = 1, \dots, m,$$

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*)^T d = 0, \quad \sum_{j=1}^m \mu_j \nabla g_j(x^*)^T d = 0,$$

$$\sum_{i=1}^k \lambda_i d^T \nabla^2 f_i(x^*) d + \sum_{j=1}^m \mu_j d^T \nabla^2 g_j(x^*) d \geq 0,$$

para todo vector $d \in \{d \in \mathbb{R}^n \mid \nabla f_i(x^*)^T d \leq 0, \forall i = 1, \dots, k, \nabla g_j(x^*)^T d \leq 0, \forall j \in J(x^*)\}$, donde $J(x^*)$ es el conjunto de índices de las restricciones activas en x^* .

Teorema 4.2.2.2 (Condición suficiente de segundo orden para la optimalidad de Pareto)

Sean las funciones objetivo del problema (4.1) funciones de clase $C^{1,1}$ y sus funciones de restricción dos veces continuamente diferenciables en un vector de decisión $x^* \in S$.

Una condición suficiente para que x^* sea óptimo de Pareto es que existan vectores $0 \leq \lambda \in \mathbb{R}^k$ y $0 \leq \mu \in \mathbb{R}^m$, tales que $(\lambda, \mu) \neq (0, 0)$ y se cumpla:

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*) + \sum_{j=1}^m \mu_j \nabla g_j(x^*) = 0,$$

$$\mu_j g_j(x^*) = 0, \quad \text{para todo } j = 1, \dots, m,$$

$$\sum_{i=1}^k \lambda_i \nabla f_i(x^*)^T d = 0, \quad \sum_{j=1}^m \mu_j \nabla g_j(x^*)^T d = 0,$$

$$\sum_{i=1}^k \lambda_i \varphi_i + d^T \left(\sum_{j=1}^m \mu_j \nabla^2 g_j(x^*) \right) d > 0,$$

para todo vector $d \in \{d \in \mathbb{R}^n \mid \nabla f_i(x^*)^T d \leq 0, \forall i = 1, \dots, k, \nabla g_j(x^*)^T d \leq 0, \forall j \in J(x^*)\}$, donde $\varphi_i \in \partial^2 f_i(x^*)(d, d)$ y $J(x^*)$ es el conjunto de índices de las restricciones activas en x^* .

5 Clasificación de los Métodos

Las soluciones del Óptimo de Pareto juegan un papel importante en la optimización multiobjetivo, tanto que se considera que un problema ya está resuelto cuando se encuentra el conjunto óptimo de Pareto.

La clasificación de los métodos se considera bajo diferentes criterios, donde en algunos se requiere la opinión de un tomador de decisiones (TDD). A continuación, tenemos la clasificación según Cohon:

- Métodos de generación: se genera un conjunto de soluciones óptimas de Pareto para que el TDD elija alguna.
- Métodos de preferencia: se consideran las preferencias del TDD para seleccionar la solución que las satisfaga.

Según Rosenthal:

- Generación parcial del conjunto óptimo de Pareto.
- Maximización explícita de una función de valor.
- Maximización implícita e interactiva de una función de valor.

Según Carmichael:

- Función objetivo compuesta.
- Función objetivo única con restricciones.
- Múltiples funciones objetivos individuales.

En este proyecto se sigue la clasificación de Hwang y Masud, basada en la participación del TDD:

1. Métodos sin preferencias.
2. Métodos a priori.
3. Métodos a posteriori.
4. Métodos interactivos.

5.1. Métodos sin Preferencias

En este método no se tiene en cuenta la opinión de un TDD. EL problema multiobjetivo se resuelve teniendo mediante un método “simple” y la solución es presentada al TDD. Este tipo de métodos es adecuado para cuando el TDD no tiene expectativas claras respecto a la solución. A continuación, se presenta uno de los métodos con esta característica.

5.1.1. Multiobjective Proximal Bundle Method (MPB)

Es una técnica avanzada de optimización diseñada para resolver problemas con múltiples objetivos no diferenciables, no convexo y sujetos a restricciones complejas, se asume que tanto las funciones objetivo y las restricciones son localmente de Lipschitz.

El MPB trata cada función objetivo individualmente manteniendo su naturaleza multiobjetivo sin reducirla a un problema uniobjetivo, es decir, este método no se utiliza directamente una función escalarizante.

Supongamos que la región factible es:

$$S = \{x \in \mathbb{R}^n \mid g(x) = (g_1(x), g_2(x), \dots, g_m(x))^T \leq 0\},$$

El método necesita combinar o comparar las distintas funciones objetivo de alguna manera para la toma de decisiones, por lo que se utiliza una función de mejora no restringida

$$H : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$$

definida por:

$$H(x^1, x^2) = \max \left[f_i(x^1) - f_i(x^2), g_l(x^1) \mid i = 1, \dots, k, l = 1, \dots, m \right].$$

La siguiente es una condición suficiente para la función mejora, se requiere que esta cumpla la calificación de restricciones de Slater.

Sea un problema de optimización multiobjetivo convexo. Se dice que cumple la **calificación de restricciones de Slater** si existe $x \in \mathbb{R}^n$ tal que:

$$g_j(x) < 0 \quad \text{para todo } j = 1, \dots, m$$

Teorema 5.1.1.1 Una condición necesaria para que un punto $x^* \in S$ sea débilmente óptimo de Pareto es que la función de mejora $H(x, x^*)$ alcance su mínimo en x^* . Si el problema es convexo y cumple la condición de Slater, entonces esta condición también es suficiente.

Algoritmo del MPB

La solución se busca de manera iterativa, hasta que se cumpla algún criterio de parada:

1. Inicialización:

- Seleccionar $x_1 \in S$.
- Definir precisión ε_s , pesos proximales u_1 , tolerancias para búsqueda lineal.

2. Dirección de búsqueda (iteración h):

- Construir modelos locales con subgradientes acumulados de funciones objetivo $f_i(x)$ y restricciones $g_i(x)$.
- Resolver el siguiente subproblema cuadrático: Encuentra una dirección d_h que minimice una combinación de mejora de objetivos y penalización proximal:

$$\min_d \{ \max_i (\alpha_{i,h} + \xi_{i,h}^T d) + \frac{u_h}{2} \|d\|^2 \}$$

donde $\alpha_{i,h}$ son estimaciones de decrecimiento y $\xi_{i,h}$ son subgradientes.

3. Búsqueda Lineal y Actualización:

- Pasos serios: Si la dirección d_h reduce significativamente todos los objetivos, actualiza $x_{h+1} = x_h + t_h d_h$.
- Pasos nulos: Si no hay mejora suficiente, refina los modelos locales sin cambiar x_h mejorando la aproximación de las funciones

4. Criterio de parada: El algoritmo termina cuando la medida de optimalidad $v_h = \max_i \alpha_{i,h}$ satisface $-\frac{1}{2}v_h < \varepsilon_s$, indicando que no se pueden mejorar más los objetivos.

Minimizar una función de mejora produce soluciones débilmente óptimas de Pareto (por Teorema) y también que cualquier solución débilmente óptima de Pareto de un problema convexo puede encontrarse bajo condiciones menores.

5.2. Métodos a Priori

Para este método el TDD debe especificar sus preferencias, expectativas y opiniones antes del proceso de solución, aunque el TDD no sabe de antemano qué es posible alcanzar en el problema. Entre uno de los métodos más conocidos se tiene el ε -constraint, que puede ser tomado tanto a priori como a posteriori.

5.2.1. ε -constraint (Restricción- ε)

Se selecciona una de las funciones objetivo para ser optimizada y las demás funciones objetivo se convierten en restricciones. El problema ε -constraint por resolver es

$$\begin{aligned} &\text{minimizar } f_l(x) \\ &\text{sujeto a } f_j(x) \leq \varepsilon_j \\ &\text{paratodo } j = 1, \dots, k, \quad j \neq l, \\ &x \in S, \text{ donde } l \in 1, \dots, k \end{aligned}$$

Teorema 5.2.1.1 La solución del problema ε -constraint es débilmente óptimo de Pareto.

Demostración: Sea $x^* \in S$ una solución del problema de ε -restricción. Supongamos que x^* no es óptimo de Pareto débil. En este caso, existe algún otro $x \in S$ tal que $f_i(x) < f_i(x^*)$ para todo $i = 1, \dots, k$. Esto significa que $f_j(x) < f_j(x^*) \leq \varepsilon_j$ para todo $j = 1, \dots, k, j \neq l$. Así, x es factible con respecto al problema de ε -restricción. Además, como $f_l(x) < f_l(x^*)$, tenemos una contradicción con la suposición de que x^* es una solución del problema de ε -restricción. Por lo tanto, x^* debe ser óptimo de Pareto débil.

Teorema 5.2.1.2 Un vector de decisión $x^* \in S$ es óptimo de Pareto si, y solo si, es solución del problema ε -restricción para cada $l = 1, \dots, k$, donde $\varepsilon_j = f_j(x^*)$ para $j = 1, \dots, k, j \neq l$.

Necesidad: Sea $x^* \in S$ óptimo de Pareto. Supongamos que no resuelve el problema de ε -restricción para algún ε donde $\varepsilon_j = f_j(x^*), j = 1, \dots, k, j \neq l$. Entonces existe una solución $x \in S$ tal que $f_l(x) < f_l(x^*)$ y $f_j(x) \leq f_j(x^*)$ cuando $j \neq l$. Esto contradice la optimalidad de Pareto de x^* . En otras palabras, x^* debe resolver el problema para cualquier función objetivo.

Suficiencia: Dado que $x^* \in S$ es, por suposición, una solución del problema de ε -restricción para cada $l = 1, \dots, k$, no existe $x \in S$ tal que $f_l(x) < f_l(x^*)$ y $f_j(x) \leq f_j(x^*)$ cuando $j \neq l$. Esta es la definición de optimalidad de Pareto para x^* .

Teorema 5.2.1.3 Un punto $x^* \in S$ es óptimo de Pareto si es la única solución del problema ε -restricción para algún l tal que $\varepsilon_j = f_j(x^*)$ para $j = 1, \dots, k, j \neq l$.

Demostración: Sea $x^* \in S$ una solución única del problema de ε -restricción para algún ε . Supongamos que no es óptimo de Pareto. En otras palabras, existe algún punto $x^0 \in S$ tal que $f_i(x^0) \leq f_i(x^*)$ para todo $i = 1, \dots, k$ y para al menos un índice j se cumple $f_j(x^0) < f_j(x^*)$.

Teorema 5.2.1.4 Sea el problema de optimización multiobjetivo convexo y sea $x^* \in S$ una solución de Pareto. Entonces, x^* es una solución de Pareto propia si y solo si el problema ε -restricción es estable para cada $l = 1, \dots, k$, donde $\varepsilon_j = f_j(x^*)$ para $j = 1, \dots, k, j \neq l$.

Algoritmo del Método

1. Selección del objetivo principal: el TDD elige una función f_j para minimizar.
2. Definición de cotas ε : se establecen valores máximos aceptables para los demás objetivos.
3. Resolución del problema uniobjetivo: se aplican algoritmos de optimización clásicos para la solución tipo simplex, gradiente proyectado...
4. Validación de preferencias: decisión satisface las preferencias se acepta, de lo contrario de deben ajustar las ε_i .

5.3. Métodos a Posteriori

Estos métodos también son llamados “métodos para generar soluciones óptimas de Pareto”. Buscan identificar un conjunto de soluciones que son presentadas al TDD, quien elige la alternativa ajustada a sus preferencias. A continuación, se mostrará uno de los métodos que hace parte de esta categoría.

5.3.1. Método de Ponderaciones (Weighting Method)

La idea de este método es transformar el problema multiobjetivo en una suma ponderada de objetivos, es decir, a cada función objetivo asociarle con un coeficiente de ponderación y minimizar la suma ponderada de los objetivos, de manera que las múltiples funciones objetivo se transforman en una sola función objetivo, este método solo captura soluciones en regiones convexas del frente de Pareto.

El problema de ponderación es de la forma:

$$\begin{aligned} & \text{minimizar } \sum_{i=1}^k w_i f_i(x) \\ & \text{sujo a } x \in S, \\ & w_i \geq 0 \text{ para todo } i = 1, \dots, k, \\ & \text{tal que } \sum_{i=1}^k w_i = 1 \end{aligned}$$

w_i son números reales, los cuales suponemos como coeficientes de ponderación. **Teorema 5.3.1.1** La solución del problema ponderado es débilmente óptima de Pareto.

Demostración: Sea $x^* \in S$ una solución del problema de ponderación. Supongamos que no es óptimo de Pareto débil. En este caso, existe una solución $x \in S$ tal que $f_i(x) < f_i(x^*)$ para todo $i = 1, \dots, k$. De acuerdo con las suposiciones establecidas sobre los coeficientes de ponderación, $w_j > 0$ para al menos un j . Entonces tenemos que $\sum_{i=1}^k w_i f_i(x) < \sum_{i=1}^k w_i f_i(x^*)$. Esto contradice la suposición de que x^* es una solución del problema de ponderación. Por lo tanto, x^* es óptimo de Pareto débil.

Teorema 5.3.1.2 La solución del problema de ponderación es óptima de Pareto si los coeficientes de ponderación son positivos, es decir, $w_i > 0$ para todo $i = 1, \dots, k$.

Demostración: Sea $x^* \in S$ una solución del problema de ponderación con coeficientes de ponderación positivos. Supongamos que no es óptimo de Pareto. Esto significa que existe una solución $x \in S$ tal que $f_i(x) \leq f_i(x^*)$ para todo $i = 1, \dots, k$ y $f_j(x) < f_j(x^*)$ para al menos un j . Dado que $w_i > 0$ para todo $i = 1, \dots, k$, tenemos que $\sum_{i=1}^k w_i f_i(x) < \sum_{i=1}^k w_i f_i(x^*)$. Esto contradice la suposición de que x^* es una solución del problema de ponderación y, por lo tanto, x^* debe ser óptimo de Pareto.

Teorema 5.3.1.3 La única solución del problema de ponderación es óptima de Pareto.

Demostración: Sea $x^* \in S$ una solución única del problema de ponderación. Supongamos que no es óptima de Pareto. En este caso, existe una solución $x \in S$ tal que $f_i(x) \leq f_i(x^*)$ para todo $i = 1, \dots, k$ y $f_j(x) < f_j(x^*)$ para al menos un j . Dado que todos los coeficientes de ponderación w_i son no negativos, tenemos $\sum_{i=1}^k w_i f_i(x) \leq \sum_{i=1}^k w_i f_i(x^*)$. Por otro lado, la unicidad de x^* implica que $\sum_{i=1}^k w_i f_i(x^*) < \sum_{i=1}^k w_i f_i(x)$ para todo $x \in S$. Las dos desigualdades anteriores son contradictorias y, por lo tanto, x^* debe ser óptima de Pareto.

Teorema 5.3.1.4 Sea el problema de optimización multiobjetivo convexo. Si $x^* \in S$ es óptimo de Pareto, entonces existe un vector de ponderación W ($w_i \geq 0, i = 1, \dots, k, \sum_{i=1}^k w_i = 1$) tal que x^* es una solución del problema de ponderación.

Todas las soluciones óptimas de Pareto de los problemas de programación lineal multiobjetivo (MOLP) pueden encontrarse mediante el método de ponderación.

Teorema 5.3.1.5 La solución del problema de ponderación es propiamente óptima de Pareto si todos los coeficientes de ponderación son positivos (condición suficiente).

Teorema 5.3.1.6 Si el problema de optimización multiobjetivo es convexo, entonces la condición suficiente también es necesaria.

Teorema 5.3.1.7 Una condición necesaria y suficiente para que un punto sea una solución de Pareto propia (óptima) de un problema de programación lineal multiobjetivo (MOLP) es que sea solución de un problema de ponderación con todos los coeficientes de ponderación positivos.

Algoritmo del Método

El método cuenta con dos fases, la generación del frente de Pareto y la selección, previo se deben haber normalizado los objetivos.

1. Generación del frente: Se resuelve el problema ponderado para múltiples vectores W distribuidos en el simplex $\sum w_i = 1$.
2. Selección a posteriori: El TDD elige una solución del conjunto generado, basándose en preferencias no articuladas inicialmente.

5.4. Métodos Interactivos

Este tipo de métodos pueden llegar a ser más efectivos, ya que el TDD puede intervenir y cooperar durante el proceso de solución del método, esto hace que el método produzca resultados más satisfactorios. Por lo que el analista trabaja junto con el TDD. Los pasos básicos en los algoritmos interactivos pueden expresarse como: a) encontrar una solución factible inicial, b) interactuar con el tomador de decisiones, y c) obtener una nueva solución (o un conjunto de nuevas soluciones). Después de un número finito de iteraciones se obtienen soluciones, si una de ellas es aceptable el método se detiene, de lo contrario, vuelve al paso b). El método que se mostrará en este trabajo es el NIMBUS, ya que es capaz de resolver problemas reales complejos.

5.4.1. Método NIMBUS

Este método combina técnicas de optimización no suave con la participación del TDD. Su objetivo es generar soluciones débilmente Pareto-óptimas adaptadas a las preferencias del tomador de decisiones. Este método funciona para problemas no convexos y funciones objetivo no diferenciables. Tiene dos versiones, la versión vector y la versión escalar.

Algoritmo del Método

1. Generar un punto de partida óptimo de Pareto, (puede usarse método de ponderación, minimizar cada función objetivo por separado, ε -constraint)

2. Pedir al tomador de decisiones que clasifique las funciones objetivo en la solución actual y que especifique niveles de aspiración y cotas superiores (si se requieren según la clasificación especificada).
3. Pedir al tomador de decisiones que seleccione el número máximo de soluciones diferentes que desea generar (entre una y cuatro) y resolver ese mismo número de subproblemas (entre los subproblemas (1)-(4)).
4. Presentar las diferentes soluciones nuevas obtenidas al tomador de decisiones.
5. Si el tomador de decisiones desea guardar una o más de las nuevas soluciones en A, incluirlas en A. Denotamos con A el conjunto de soluciones guardadas. Inicialmente, A se define como el conjunto vacío: $A = \emptyset$.
6. Si el tomador de decisiones no desea ver soluciones intermedias entre dos soluciones, pasar al paso 8. De lo contrario, pedirle que seleccione las dos soluciones entre las nuevas soluciones o las soluciones en A. Pedir también el número de soluciones intermedias deseadas.
7. Generar el número deseado de soluciones intermedias y proyectarlas sobre el conjunto de soluciones óptimas de Pareto. Ir al paso 4.
8. Pedir al tomador de decisiones que elija la solución más preferida entre las nuevas soluciones, las intermedias y/o las soluciones en A. Denotarla como la solución actual. Si el tomador de decisiones desea continuar, volver al paso 2. De lo contrario, terminar.
9. El algoritmo se detiene si el tomador de decisiones no desea disminuir ningún valor objetivo o no está dispuesto a permitir que aumente alguno de ellos. De lo contrario, la búsqueda continúa iterativamente desplazándose dentro del conjunto de soluciones óptimas de Pareto.

6 El problema de Markowitz

La teoría moderna de portafolios, propuesta por Markowitz (1952), constituye uno de los fundamentos más importantes de la finanza cuantitativa. Su modelo describe cómo los inversionistas racionales pueden construir portafolios que maximizan el retorno esperado para un nivel dado de riesgo, o alternativamente, minimizan el riesgo para un nivel dado de retorno esperado.

Desde una perspectiva de optimización multiobjetivo, este problema se puede formular como la necesidad de encontrar un compromiso entre dos objetivos que son naturalmente conflictivos:

- Maximizar el retorno esperado de un portafolio.
- Minimizar el riesgo del portafolio, usualmente medido mediante la varianza de los retornos.

Esto da lugar a una frontera de soluciones conocidas como la frontera eficiente (efficient frontier), compuesta por todos los portafolios que no pueden mejorar en un objetivo sin empeorar el otro.

Formulación Supongamos que se tienen n activos financieros, y sea

- $x = [x_1, x_2, \dots, x_n]^T$ es el vector de proporciones de inversión en cada activo x_i .
- $\mu = [\mu_1, \mu_2, \dots, \mu_n]^T$ es el vector de retornos esperados de los activos

- Σ es la matriz de covarianza

Entonces, el problema de Markowitz en su versión multiobjetivo puede ser formulado como:

$$\begin{aligned} \text{Minimizar } f_1(x) &= \sigma^2(x) = x^T \Sigma x && (\text{varianza: riesgo}) \\ \text{Maximizar } f_2(x) &= \mu^T x && (\text{Retorno esperado}) \end{aligned}$$

Una vez obtenida la frontera eficiente de portafolios mediante el modelo de Markowitz, surge la necesidad de seleccionar una única solución “óptima” entre todas las posibles combinaciones riesgo-retorno. Para esto, una de las métricas más utilizadas en finanzas es el coeficiente de Sharpe, propuesto por William F. Sharpe en 1966.

El coeficiente de Sharpe mide el exceso de retorno por unidad de riesgo asumido y se define como:

$$S(x) = \frac{R_p - r_f}{\sigma_p}$$

Donde:

- R_p : retorno esperado del portafolio
- r_f : tasa libre de riesgo
- σ_p : desviación estándar del portafolio

En la práctica, es común introducir ciertos supuestos o modificaciones al modelo clásico de Markowitz para facilitar la interpretación de resultados o adaptarse a condiciones específicas del mercado. A continuación presentamos dos ajustes importantes que adoptaremos en este trabajo:

■ Asunción de la tasa libre de riesgo nula

A efectos del análisis, se asumirá que la tasa libre de riesgo es igual a cero, es decir:

$$r_f = 0$$

Bajo esta suposición, el coeficiente de Sharpe se simplifica a:

$$S(x) = \frac{R_p}{\sigma_p}$$

Esto implica que se mide el retorno total del portafolio por unidad de riesgo, sin referencia a un activo sin riesgo. Esta aproximación es razonable en contextos donde no existe un activo verdaderamente libre de riesgo o cuando se desea evaluar portafolios desde una perspectiva puramente relativa.

■ Minimización de la desviación estándar en lugar de la varianza

Si bien el modelo original de Markowitz utiliza la varianza como medida del riesgo, en este trabajo se opta por minimizar la desviación estándar del portafolio, definida como:

$$\sigma_p = \sqrt{x^T \Sigma x}$$

La razón principal de este cambio es que la desviación estándar, a diferencia de la varianza, está expresada en las mismas unidades que el retorno, lo cual facilita la comparación directa y mejora la visualización de los resultados en los gráficos de frontera eficiente. En particular, el eje horizontal de dicho gráfico representará el riesgo como desviación estándar, que es más intuitiva.

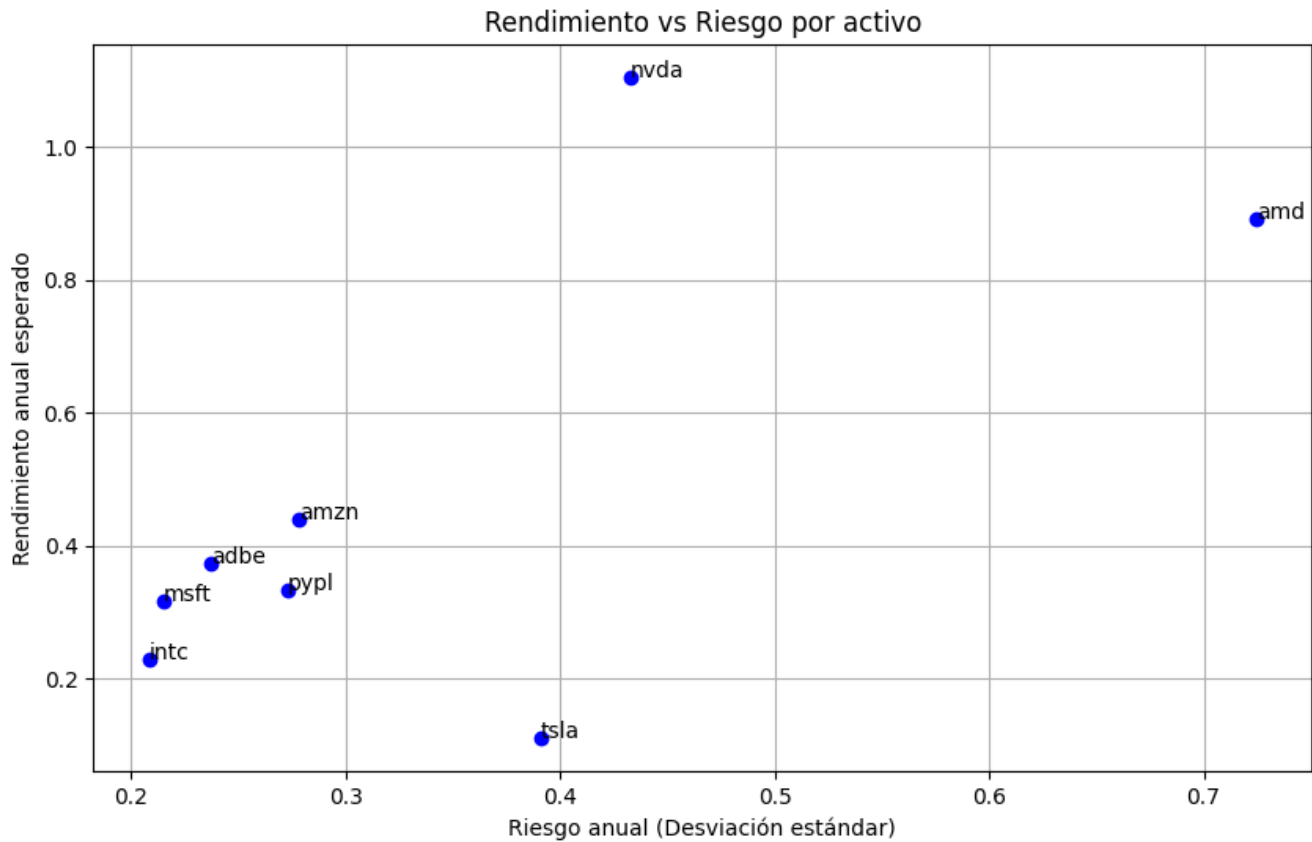
Con el objetivo de aplicar los conceptos desarrollados anteriormente, se llevó a cabo un ejercicio práctico de construcción de portafolios eficientes utilizando datos reales del mercado accionario.

Los datos utilizados fueron extraídos del conjunto disponible en Kaggle titulado Huge Stock Market Dataset. Este dataset contiene precios históricos diarios de cientos de activos financieros que cotizan en el mercado estadounidense.

Para este ejercicio se seleccionaron los siguientes ocho activos de gran capitalización, pertenecientes al sector tecnológico:

- AMD (amd)
- Amazon (amzn)
- Nvidia (nvda)
- Tesla (tsla)
- Paypal (pypl)
- Intel (intc)
- Adobe (adbe)
- Microsoft (msft)

Se consideraron sus precios ajustados de cierre durante un periodo histórico común para todos los activos, a partir del cual se calcularon los retornos diarios y la matriz de covarianzas.



Método 1: método de Monte Carlo

El método de Monte Carlo es una técnica que utiliza simulación aleatoria para resolver problemas matemáticos o físicos complejos. La idea principal es generar una gran cantidad de números aleatorios para aproximar una solución cuando no es posible (o es muy difícil) obtenerla de forma exacta.

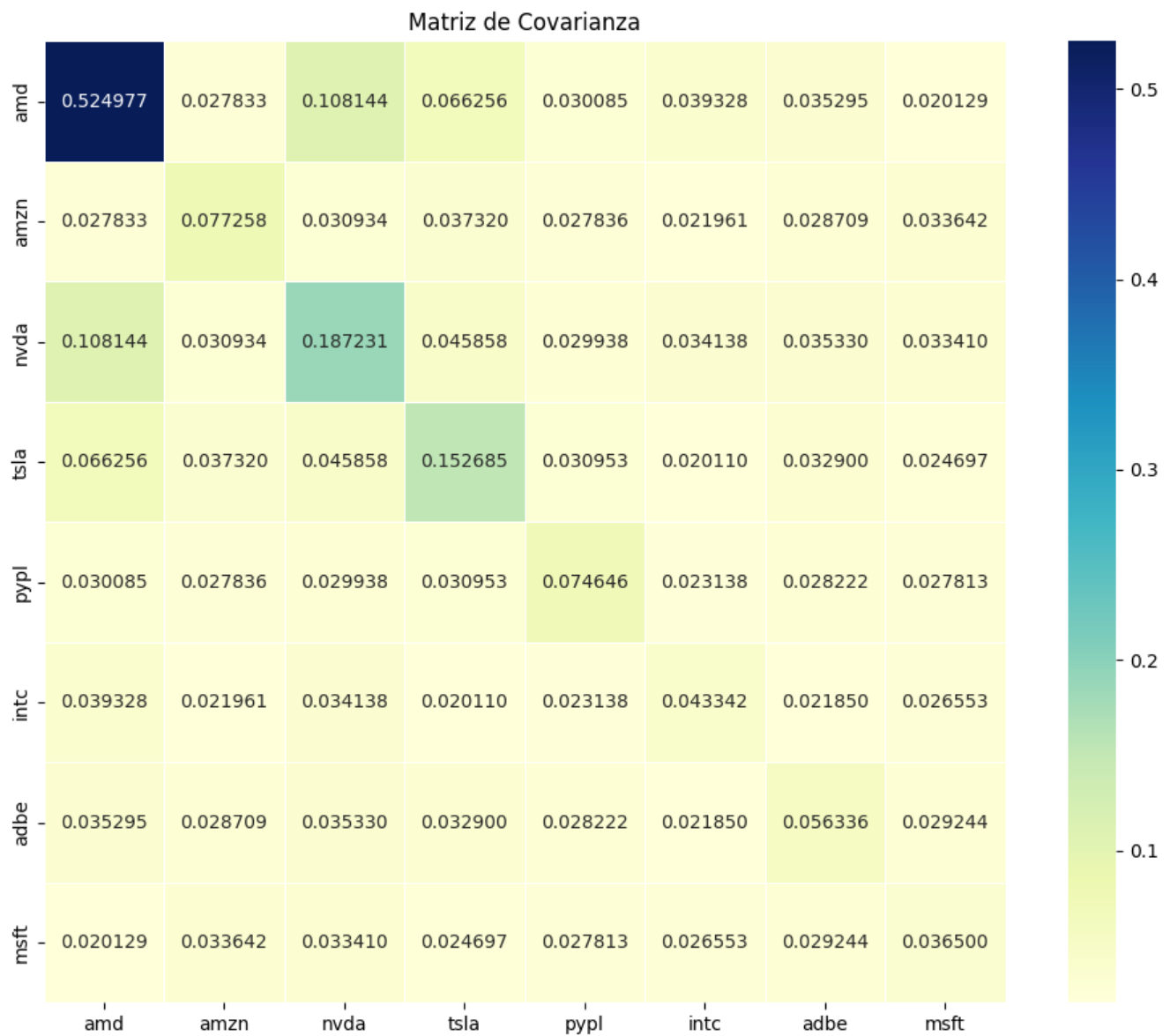
Se generó una gran cantidad de portafolios aleatorios (del orden de 10,000) asignando pesos aleatorios a los activos, bajo las siguientes restricciones:

- Los pesos deben sumar 1 (portafolio completamente invertido)
- No se permite venta en corto ($x_i \geq 0$)

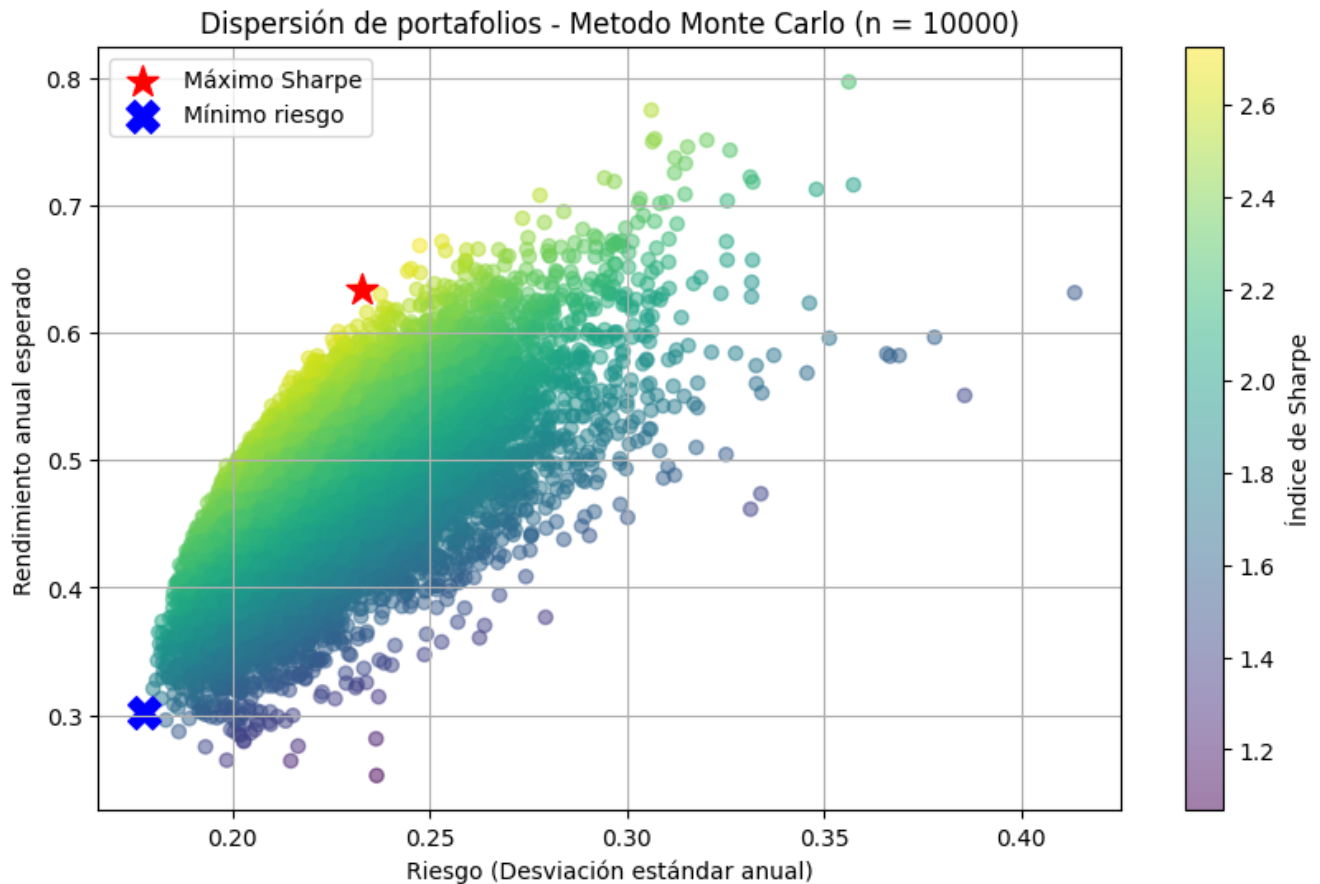
Para cada portafolio se calcularon su retorno esperado, su desviación estándar y su coeficiente de Sharpe. Esta técnica permite visualizar de forma exploratoria la distribución de posibles combinaciones riesgo-retorno.

Portafolio Maximo Sharpe (Monte Carlo):

Índice de Sharpe del portafolio: 2.7247



Activo	Proporción portafolio
amd	3.47%
amzn	25.24%
nvda	32.15%
tsla	0.14%
pypl	11.22%
intc	1.36%
adbe	22.51%
msft	3.92%



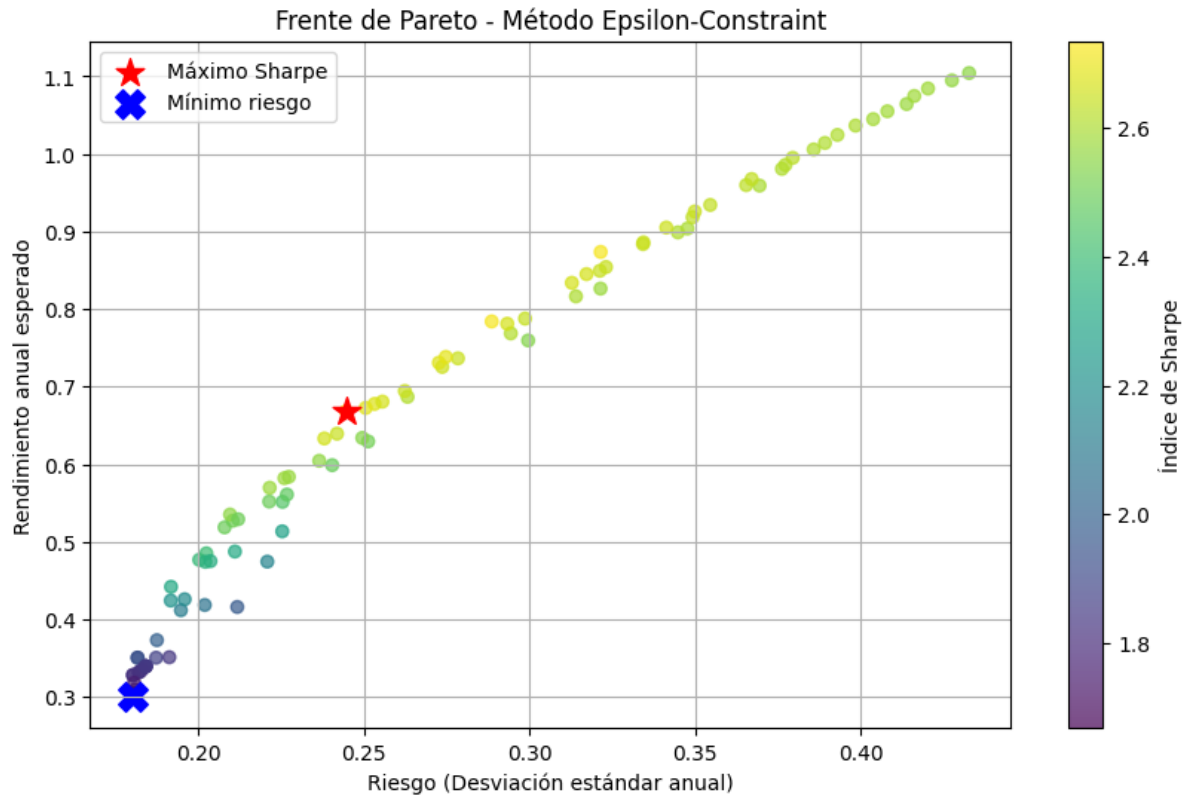
Método 2: método de ε -constraint

En esta estrategia, se resolvió un problema de optimización multiobjetivo utilizando el enfoque ε -constraint. Para ello se utilizó el paquete pymoo, especializado en optimización multiobjetivo en Python.

El procedimiento consistió en:

- Minimizar el riesgo (desviación estándar del portafolio)
- Sujetar el retorno a un valor mínimo ε , variando sistemáticamente dicho parámetro

Este barrido de valores de ε permite construir una aproximación al frente de Pareto, donde cada punto representa un portafolio eficiente que ofrece el mínimo riesgo posible para un nivel de retorno específico.



Portafolio Máximo Sharpe (ε -constraint):

Índice de Sharpe del portafolio: 2.7332

Activo	Proporción portafolio
amd	0.61%
amzn	21.81%
nvda	40.14%
tsla	0.16%
pypl	5.36%
intc	2.48%
adbe	11.82%
msft	17.62%

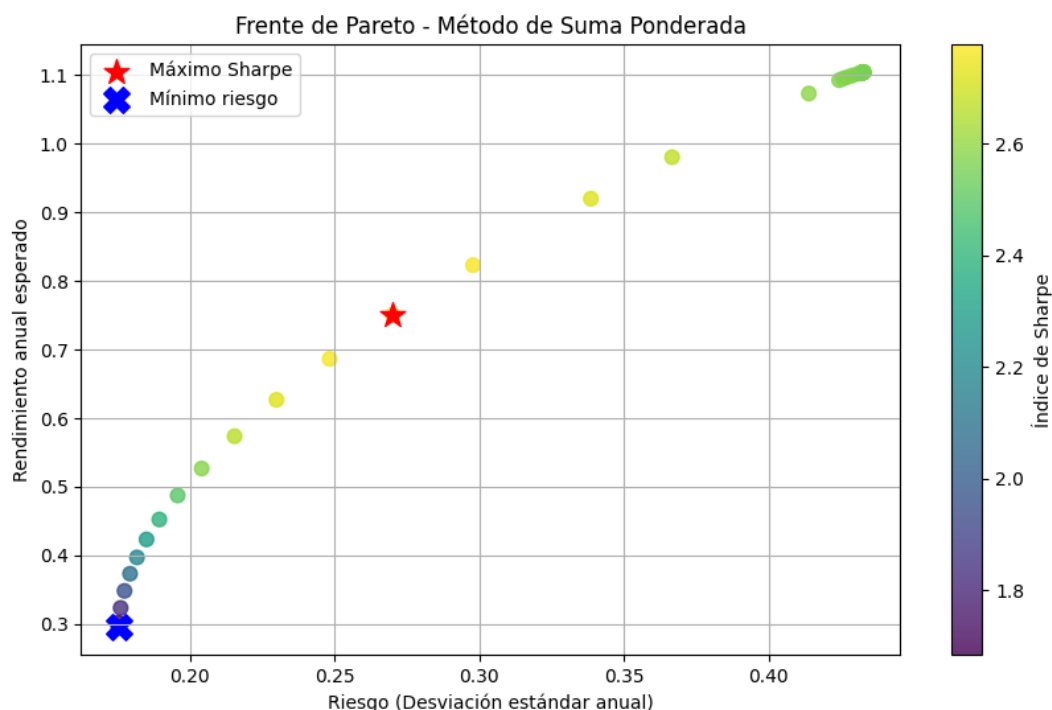
6.1. Método 3: método de suma ponderada

Como tercera estrategia se utilizó el método de suma ponderada de objetivos, también conocido como weighted sum. En este caso, el problema multiobjetivo se convierte en un único problema escalar al minimizar la función:

$$F_{\lambda}(x) = \lambda \cdot \sigma_p + (1 - \lambda) \cdot (-R_p)$$

Donde $\lambda \in [0, 1]$ regula la importancia relativa entre riesgo y retorno. Para resolver este problema, se utilizó la función minimize del paquete scipy.optimize, haciendo un barrido de valores de λ entre 0 y 1.

Cada valor de λ genera una solución distinta, lo que permite construir un frente de Pareto aproximado.



Portafolio Máximo Sharpe (Suma ponderada)

Índice de Sharpe del portafolio: 2.7773

Activo	Proporción portafolio
amd	4.05%
amzn	26.91%
nvda	46.61%
tsla	0.00%
pypl	7.43%
intc	0.00%
adbe	15.00%
msft	0.00%

Conclusiones

A continuación se presentan las conclusiones principales obtenidas a partir del análisis de optimización de portafolios utilizando tres métodos: Monte Carlo, Epsilon-Constraint y Suma Ponderada. El criterio de evaluación fue el índice de Sharpe.

1. El portafolio más eficiente lo genera el método de Suma Ponderada.

Este método alcanzó el mayor índice de Sharpe: 2.7773, superando al método de Monte Carlo (2.7247) y al Epsilon-Constraint (2.7332). Aunque las diferencias numéricas son pequeñas, pueden traducirse en mejoras significativas en el rendimiento ajustado por riesgo.

2. El método Suma Ponderada concentra la inversión en pocos activos.

El portafolio óptimo resultante asigna más del 88 % del capital a tres activos: nvda (46.61 %), amzn

(26.91 %), y adbe (15 %), dejando por fuera activos como tsla, intc y msft. Esto puede implicar mayor riesgo por falta de diversificación.

3. **Epsilon-Constraint y Monte Carlo generan portafolios más diversificados.**

Ambos métodos tienden a distribuir la inversión entre más activos, lo que puede traducirse en una mejor cobertura ante la volatilidad del mercado. Por ejemplo, el portafolio óptimo de Monte Carlo asigna pesos distintos a los ocho activos considerados, aunque algunos son bajos.

4. **El método Epsilon-Constraint presenta un costo computacional elevado.**

La generación del frente de Pareto mediante este enfoque tomó **varios minutos**, en comparación con los métodos de Monte Carlo y Suma Ponderada, cuyos resultados se obtuvieron de forma casi inmediata. Esto representa una desventaja en contextos que requieren eficiencia computacional.

5. **Comparación general de los métodos utilizados:**

Criterio	Monte Carlo	Epsilon-Constraint	Suma Ponderada
Índice de Sharpe máximo	2.7247	2.7332	2.7773
Diversificación	Alta	Media	Baja
Tiempo de ejecución	Rápido	Lento (minutos)	Rápido
Tipo de búsqueda	Aleatoria	Determinística	Determinística
Facilidad para obtener el frente	Baja	Alta	Alta

Referencias

- [BD20] J. Blank and K. Deb. pymoo: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509, 2020.
- [BF17] Rasmus Bokrantz and Albin Fredriksson. Necessary and sufficient conditions for pareto efficiency in robust multiobjective optimization. *European Journal of Operational Research*, 262(2):682–692, 2017.
- [MA04] R. T. Marler and J. S. Arora. Survey of multi-objective optimization methods for engineering. *Structural and Multidisciplinary Optimization*, 26(6):369–395, Abril 2004.
- [Mar52] Harry M. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [Mar17] Boris Marjanovic. Huge stock market dataset. <https://www.kaggle.com/datasets/borismarjanovic/price-volume-data-for-all-us-stocks-etfs>, 2017. Accedido el 20 de mayo de 2025.
- [Mie99] Kaisa Miettinen. *Nonlinear Multiobjective Optimization*, volume 12 of *International Series in Operations Research & Management Science*. Kluwer Academic Publishers, Boston, 1999.
- [MM95] Kaisa Miettinen and Marko M. Mäkelä. Interactive bundle-based method for nondifferentiable multiobjective optimization: Nimbus. *Optimization*, 34(3):231–246, 1995.
- [Sha66] William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1, Part 2: Supplement on Security Prices):119–138, 1966.

Anexo: Código

```
1 !pip install pymoo
2
3 import pandas as pd
4 import numpy as np
5 import seaborn as sns
6 import matplotlib.pyplot as plt
7 from pymoo.core.problem import ElementwiseProblem
8 from pymoo.algorithms.soo.nonconvex.de import DE
9 from pymoo.constraints.eps import AdaptiveEpsilonConstraintHandling
10 from pymoo.optimize import minimize
11 from pymoo.problems.single import G1
12 from pymoo.termination import get_termination
13 from scipy.optimize import minimize as minimize2
14
15 from google.colab import drive
16 drive.mount ("/content/drive")
17
18 symbols = ['amd', 'amzn', 'nvda', 'tsla', 'pypl', 'intc', 'adbe', 'msft']
19 data = {}
20 loaded_symbols = []
21
22 ## Tratamiento de datos
23 for symbol in symbols:
24     try:
25         df = pd.read_csv(
26             f"/content/drive/MyDrive/Colab Notebooks/optimalidad de pareto/{symbol}.us.txt")
27         df['Date'] = pd.to_datetime(df['Date'])
28         df.set_index('Date', inplace=True)
29
30         # Verifica que la columna 'Close' existe y tiene datos válidos
31         if 'Close' in df.columns and not df['Close'].isnull().all():
32             data[symbol] = df['Close']
33             loaded_symbols.append(symbol)
34         else:
35             print(f" {symbol}: columna 'Close' vacía o no válida.")
36
37     except Exception as e:
38         print(f"Error al cargar {symbol}: {e}")
39
40 # Concatenar usando el diccionario, que asigna nombres automáticamente
41 prices = pd.concat(data, axis=1).dropna().sort_index()
42
43 print(f" Activos cargados: {loaded_symbols}")
44 print(prices.head())
45
46 returns = prices.pct_change().dropna()
47
```

```

48 mean_returns = returns.mean()
49 risk = returns.std()
50
51
52 trading_days = 252 # número típico de días bursátiles al año
53 annual_returns = mean_returns * trading_days
54 annual_risks = risk * (trading_days ** 0.5)
55
56 ## Rendimiento vs riesgo por activo
57
58 plt.figure(figsize=(10, 6))
59 plt.scatter(annual_risks, annual_returns, c='blue')
60 for i, symbol in enumerate(annual_returns.index):
61     plt.annotate(symbol, (annual_risks[i], annual_returns[i]))
62 plt.xlabel("Riesgo anual (Desviación estándar)")
63 plt.ylabel("Rendimiento anual esperado")
64 plt.title("Rendimiento vs Riesgo por activo")
65 plt.grid(True)
66 plt.show()
67
68 ## Metodo Monte Carlo
69 num_portfolios = 10000
70
71 annual_returns = returns.mean() * 252
72 annual_risks = returns.std() * np.sqrt(252)
73 cov_matrix = returns.cov() * 252
74
75 print(cov_matrix)
76 results = {'ret': [], 'risk': [], 'sharpe': [], 'weights': []}
77
78 for _ in range(num_portfolios):
79     weights = np.random.random(len(symbols))
80     weights /= np.sum(weights)
81
82     port_return = np.dot(weights, annual_returns)
83     port_risk = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
84     sharpe_ratio = port_return / port_risk # Sin tasa libre de riesgo
85
86     results['ret'].append(port_return)
87     results['risk'].append(port_risk)
88     results['sharpe'].append(sharpe_ratio)
89     results['weights'].append(weights)
90
91 # Convertimos a arrays
92 returns_arr = np.array(results['ret'])
93 risks_arr = np.array(results['risk'])
94 sharpes_arr = np.array(results['sharpe'])
95
96 # Índices de portafolios clave
97 max_sharpe_idx = sharpes_arr.argmax()
98 min_risk_idx = risks_arr.argmin()

```



```

99
100 # Graficamos
101 plt.figure(figsize=(10, 6))
102 plt.scatter(risks_arr, returns_arr, c=sharpes_arr, cmap='viridis', alpha=0.5)
103 plt.colorbar(label='Índice de Sharpe')
104 plt.xlabel('Riesgo (Desviación estándar anual)')
105 plt.ylabel('Rendimiento anual esperado')
106 plt.title('Dispersión de portafolios - Metodo Monte Carlo (n = 10000)')
107
108 # Punto Sharpe máximo
109 plt.scatter(risks_arr[max_sharpe_idx], returns_arr[max_sharpe_idx],
110             marker='*', color='red', s=200, label='Máximo Sharpe')
111
112 # Punto de menor riesgo
113 plt.scatter(risks_arr[min_risk_idx], returns_arr[min_risk_idx],
114             marker='X', color='blue', s=200, label='Mínimo riesgo')
115
116 plt.legend()
117 plt.grid(True)
118 plt.show()
119
120 optimal_weights = results['weights'][max_sharpe_idx]
121 for symbol, weight in zip(symbols, optimal_weights):
122     print(f"{symbol}: {weight:.2%}")
123
124 max_sharpe_value = sharpes_arr[max_sharpe_idx]
125 print(f"Índice de Sharpe máximo: {max_sharpe_value:.4f}")
126
127
128 ## Metodo Epsilon - Constraint
129 returns = np.array([0.892037, 0.439972, 1.104205, 0.110369, 0.334007, 0.230357, 0.373887, 0.316870])
130 cov_matrix = np.array([
131     [0.524977, 0.027833, 0.108144, 0.066256, 0.030085, 0.039328, 0.035295, 0.020129],
132     [0.027833, 0.077258, 0.030934, 0.037320, 0.027836, 0.021961, 0.028709, 0.033642],
133     [0.108144, 0.030934, 0.187231, 0.045858, 0.029938, 0.034138, 0.035330, 0.033410],
134     [0.066256, 0.037320, 0.045858, 0.152685, 0.030953, 0.020110, 0.032900, 0.024697],
135     [0.030085, 0.027836, 0.029938, 0.030953, 0.074646, 0.023138, 0.028222, 0.027813],
136     [0.039328, 0.021961, 0.034138, 0.020110, 0.023138, 0.043342, 0.021850, 0.026553],
137     [0.035295, 0.028709, 0.035330, 0.032900, 0.028222, 0.021850, 0.056336, 0.029244],
138     [0.020129, 0.033642, 0.033410, 0.024697, 0.027813, 0.026553, 0.029244, 0.036500],
139 ])
140
141 # clase del problema pymoo
142 class PortfolioOptimization(ElementwiseProblem):
143     def __init__(self, epsilon_return):
144         n_assets = len(returns)
145         super().__init__(n_var=n_assets, n_obj=1, n_ieq_constr=1, n_eq_constr=1, xl=0.0, xu=1.0)
146         self.epsilon_return = epsilon_return
147
148     def _evaluate(self, x, out, *args, **kwargs):
149         x_norm = x / np.sum(x)

```

```

150     risk = np.dot(x_norm.T, np.dot(cov_matrix, x_norm))
151     expected_return = np.dot(x_norm, returns)
152
153     out["F"] = risk
154     out["G"] = [self.epsilon_return - expected_return] # - retorno 0
155     out["H"] = np.sum(x) - 1
156
157     # Parametrización del algoritmo
158     algorithm = DE(pop_size=100)
159     termination = get_termination("n_gen", 200)
160
161     # Barrido sobre distintos valores de epsilon (retorno mínimo)
162     epsilons = np.linspace(min(returns), max(returns), 100)
163
164     risks_arr = []
165     returns_arr = []
166     weights_arr = []
167
168     for eps in epsilons:
169         problem = PortfolioOptimization(epsilon_return=eps)
170
171         res = minimize(problem,
172                       algorithm,
173                       termination,
174                       seed=42,
175                       verbose=False)
176
177         if res.F is not None:
178             weights = res.X / np.sum(res.X)
179             risk = float(res.F)
180             expected_return = float(np.dot(weights, returns))
181
182             risks_arr.append(np.sqrt(risk)) # desviación estándar
183             returns_arr.append(expected_return)
184             weights_arr.append(weights)
185
186     # Convertimos a arrays
187     risks_arr = np.array(risks_arr)
188     returns_arr = np.array(returns_arr)
189     weights_arr = np.array(weights_arr)
190
191     # Encontrar índice del Sharpe máximo (asumimos rf = 0)
192     sharpes_arr = returns_arr / risks_arr
193     max_sharpe_idx = np.argmax(sharpes_arr)
194     min_risk_idx = np.argmin(risks_arr)
195
196     # Gráfica
197     plt.figure(figsize=(10, 6))
198     plt.scatter(risks_arr, returns_arr, c=sharpes_arr, cmap='viridis', alpha=0.7)
199     plt.colorbar(label='Índice de Sharpe')
200     plt.xlabel('Riesgo (Desviación estándar anual)')

```

```

201 plt.ylabel('Rendimiento anual esperado')
202 plt.title('Frente de Pareto - Método Epsilon-Constraint')
203
204 # Punto Sharpe máximo
205 plt.scatter(risks_arr[max_sharpe_idx], returns_arr[max_sharpe_idx],
206             marker='*', color='red', s=200, label='Máximo Sharpe')
207
208 # Punto de menor riesgo
209 plt.scatter(risks_arr[min_risk_idx], returns_arr[min_risk_idx],
210             marker='X', color='blue', s=200, label='Mínimo riesgo')
211
212 plt.legend()
213 plt.grid(True)
214 plt.show()
215
216 # Mostrar pesos óptimos del máximo Sharpe
217 optimal_weights = weights_arr[max_sharpe_idx]
218 for symbol, weight in zip(symbols, optimal_weights):
219     print(f"{symbol}: {weight:.2%}")
220
221 print(f"\nÍndice de Sharpe máximo: {sharpes_arr[max_sharpe_idx]:.4f}")
222
223 returns1 = returns_arr
224 risks1 = risks_arr
225 sharpes1 = sharpes_arr
226 weights1 = weights_arr
227
228 ## Metodo suma ponderada
229
230 def weighted_sum_objective(weights, returns, cov_matrix, lambda_):
231     risk = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
232     expected_return = np.dot(weights, returns)
233     return lambda_ * risk - (1 - lambda_) * expected_return
234
235 n_assets = len(returns)
236
237 # Restricción: suma de pesos = 1
238 constraints = {'type': 'eq', 'fun': lambda x: np.sum(x) - 1}
239
240 # Límite de cada peso: entre 0 y 1
241 bounds = tuple((0, 1) for _ in range(n_assets))
242
243 # Valor inicial: pesos uniformes
244 x0 = np.array([1.0 / n_assets] * n_assets)
245
246 lambdas = np.linspace(0, 1, 50)
247
248 risks_ws = []
249 returns_ws = []
250 solutions_ws = []
251

```

```

252 for lambda_ in lambdas:
253     res = minimize2(weighted_sum_objective,
254                     x0=x0,
255                     args=(returns, cov_matrix, lambda_),
256                     method='SLSQP',
257                     bounds=bounds,
258                     constraints=constraints)
259
260     if res.success:
261         w = res.x
262         risk = np.sqrt(np.dot(w.T, np.dot(cov_matrix, w)))
263         ret = np.dot(w, returns)
264
265         risks_ws.append(risk)
266         returns_ws.append(ret)
267         solutions_ws.append(w)
268     else:
269         print(f"Falló la optimización para lambda = {lambda_: .2f}")
270
271 sharpes = [r / s for r, s in zip(returns_ws, risks_ws)]
272
273 sharpes = [r / s for r, s in zip(returns_ws, risks_ws)]
274 max_sharpe_idx = np.argmax(sharpes)
275 min_risk_idx = np.argmin(risks_ws)
276
277 plt.figure(figsize=(10, 6))
278 scatter = plt.scatter(risks_ws, returns_ws, c=sharpes, cmap='viridis', s=60, alpha=0.8)
279 plt.colorbar(scatter, label='Índice de Sharpe')
280
281 plt.xlabel('Riesgo (Desviación estándar anual)')
282 plt.ylabel('Rendimiento anual esperado')
283 plt.title('Frente de Pareto - Método de Suma Ponderada')
284 plt.grid(True)
285
286 # Máximo Sharpe
287 plt.scatter(risks_ws[max_sharpe_idx], returns_ws[max_sharpe_idx],
288             marker='*', color='red', s=200, label='Máximo Sharpe')
289
290 # Mínimo riesgo
291 plt.scatter(risks_ws[min_risk_idx], returns_ws[min_risk_idx],
292             marker='X', color='blue', s=200, label='Mínimo riesgo')
293
294 plt.legend()
295 plt.show()
296
297 sharpes = [r / s for r, s in zip(returns_ws, risks_ws)]
298 max_idx = np.argmax(sharpes)
299
300 print("Portafolio con mayor índice de Sharpe:")
301 for symbol, weight in zip(symbols, solutions_ws[max_idx]):
302     print(f"{symbol}: {weight:.2%}")

```

```

303 print(f"Índice de Sharpe: {sharpes[max_idx]:.4f}")
304
305 returns2 = returns_ws
306 risks2 = risks_ws
307 sharpes2 = sharpes
308 weights2 = solutions_ws
309
310 ## Matriz de covarianza
311 cov_matrix = np.array([
312     [0.524977, 0.027833, 0.108144, 0.066256, 0.030085, 0.039328, 0.035295, 0.020129],
313     [0.027833, 0.077258, 0.030934, 0.037320, 0.027836, 0.021961, 0.028709, 0.033642],
314     [0.108144, 0.030934, 0.187231, 0.045858, 0.029938, 0.034138, 0.035330, 0.033410],
315     [0.066256, 0.037320, 0.045858, 0.152685, 0.030953, 0.020110, 0.032900, 0.024697],
316     [0.030085, 0.027836, 0.029938, 0.030953, 0.074646, 0.023138, 0.028222, 0.027813],
317     [0.039328, 0.021961, 0.034138, 0.020110, 0.023138, 0.043342, 0.021850, 0.026553],
318     [0.035295, 0.028709, 0.035330, 0.032900, 0.028222, 0.021850, 0.056336, 0.029244],
319     [0.020129, 0.033642, 0.033410, 0.024697, 0.027813, 0.026553, 0.029244, 0.036500],
320 ])
321
322 symbols = ['amd', 'amzn', 'nvda', 'tsla', 'pypl', 'intc', 'adbe', 'msft']
323 cov_df = pd.DataFrame(cov_matrix, index=symbols, columns=symbols)
324
325 plt.figure(figsize=(10, 8))
326 sns.heatmap(cov_df, annot=True, fmt=".6f", cmap="YlGnBu", linewidths=0.5, square=True)
327 plt.title("Matriz de Covarianza")
328 plt.tight_layout()
329 plt.show()

```